A package for PLATE RECognition In PYthon

Pejvak Javaheri

Version 1.0.0, documentation updated on June 5, 2025

Contents

1 Introduction

PLATE RECognition In PYthon (`platerecipy`) is a package tailored for plate identification on 3D spherical surface of mantle convection models and geodetic datasets[1]. At its core, `platerecipy` uses Random Walker (RW) algorithm developed by Grady (2006) but modified for a spherical surface. **The probability array obtained from RW is the main feature of this package that enables further quantitative analysis of the identified plates.**

2 Dependencies

Currently, Linux is the only supported platform. Since MacOS is Unix-based, `platerecipy` is expected to be installed and used without a problem. The main reason that Windows is not supported at this point is that there some functionalities are implemented in `C` and require a slightly different compilation and linking procedure. Until the first release, availability of `platerecipy` on Windows is not a development priority.

Concerning the `Python` dependencies, `pip` should automatically install the necessary packages. Nonetheless, the following includes the necessary dependencies:

- `numpy` (Harris et al., 2020): for array operations
- `scipy` (Virtanen et al., 2020): for specific scientific utilities
- `matplotlib` (Hunter, 2007): for basic plots
- `pyvista` (Sullivan & Kaszynski, 2019): for `vtk` output and 3D visualization
- `pandas` (pandas development team, 2020): for additional io operations

3 Installation

Until the packages is released on PyPI, `platerecipy` can be installed using `pip` on the shell as follows:

```
cd dist/
python -m pip install platerecipy-?.?.?.tar.gz
```

where `platerecipy-?.?.?.tar.gz` is the installation tar file for the desired version. Note that all installation tar files can be found in `dist` subdirectory.

4 Basic interface

For most purposes, only the following modules are of interest:

- `grid`: providing class and function definitions to interpolate and construct a grid,
- `model`: providing class and function definitions to defined a plate model and identify candidate plates, andthese

[1] Though most `platerecipy` modules support a 2D input, additional work remains for a full integration of such functionalities.

- `io`: providing helper functions for outputting in various formats.

Accessing these modules are done simply using `import`:

```
from platerecipy.grid import SphericalGrid

from platerecipy.model import PlateModel

from platerecipy.io import save_as_mat, save_as_vtp, save_as_csv
```

(though discouraged, one can import all the content by * wildcard).

`platerecipy` is developed under the notion that it must accommodate users with different dataset formats and from different grid structures (e.g., spherical, YinYang, icosahedral, etc.). Accordingly, as much as possible, the interface must refrain from assuming any format or structure specific layout to the input arguments. Consequently, the trade-off of optimizing for minimal conversion burden on the end user is more processing and higher memory usage. However, this extra cost at the post processing is vanishingly small compared to the original cost of running numerical models.

With the aforementioned principle in mind, all that is expected from the user is to have access to, at least, the following arrays:

- the Cartesian x, y, and z coordinates (will refer to it as `input_xs`, `input_ys`, and `input_zs`),
- as well as the corresponding field values (will refer to it as `input_field`).

Furthermore, though all input arrays are assumed to be `numpy` (Harris et al., 2020) arrays, they can have any shape (flat or multidimensional).

The fist step is to generate a grid:

```
from platerecipy.grid import SphericalGrid

# generating a consistent grid for interpolation
grid = SphericalGrid(input_xs, input_ys, input_zs)
```

The `SphericalGrid` object is itself a derived type of a generic class `Grid`, but it informs the code whether additional considerations (e.g., applying the wraparound boundary condition or spherical distances) are required. Then, our `SphericalGrid` object is to be used to interpolate `input_field` to the input points to a spherical grid. Segmentation is performed on the linearly-interpolated uniform spherical grid. Interpolation process is performed using the method `interpolate_field`:

```
# interpolating an input field
field = grid.interpolate_field(input_field)
```

It should be noted that for input fields such as viscosity or strain-rate (which vary by orders of magnitude), the optional argument `take_log` should be set to `True` in order to the linear interpolation is performed in the log space (and then converted back).

With the `grid` object and the interpolated field `field`, we have the necessary arguments to perform plate detection. To initiate the process, a `PlateModel` object is to be constructed as follows:

```
from platerecipy.model import PlateModel

# initializing a plate model
m = PlateModel(grid)
```

The newly-created object `m` carries (a pointer) to the `grid` object which includes information about the original input as well as the interpolated grid structure. Though it would have been possible to keep interpolation under the hood (given `PlateModel` carries a `Grid` attribute), the user may want to further manipulate the interpolated field before passing it for plate detection. That is why the two steps are kept separate. The importance of the class `PlateModel` is in the fundamental idea that what defines plate interior/boundary must be made explicit and consistent across subsequent snapshots of the same model. The `PlateModel` class has the ability to stack several fields with different corresponding weights and normalize the stacked field to span $[0, 1]$. Stacking fields are done through the `PlateModel.stack_field()` method as follows:

```
# stacking the interpolated field
m.stack_field(field)
```

Once again, for fields that vary by orders of magnitude, `PlateModel.stack_field()` accepts an optional argument `take_log` which if set to `True` it takes the logarithm to flatten the distribution. Most importantly, the stacked fields are assumed to be markers of deformation such that 0 and 1 correspond to minimal and maximal deformation. For a field that is opposite of this (i.e., its high corresponds to low deformation and vice versa), there is an additional argument `invert` which ensures the stacked field stays consistent.

Once all the fields (if multiple) are stacked, the `PlateModel.find_plates()` method performs segmentation:

```
# finding plates on the stacked field
plate_IDs = m.find_plates(
    boundary_quantile  = 0.9,          # threshold for the boundaries
    # ...
)
```

The `PlateModel.find_plates()` method both returns the segmentation result, and also it save them in the model object as `m.plate_IDs` (containing segmentation IDs) and `m.ID_probs` (containing the associated probabilities).

5 Examples

In this section, a number of examples are provided to demonstrate various features of `platerecipy`.

5.1 `examples/basic.py`: A template from input to `.vtp` output

```
"""
@file basic.py
@author Pejvak Javaheri; pejvak.javaheri@mail.utoronto.ca
@brief A template for basic usage of platerecipy.
"""

# ~~~~~~~~~~~~~~~ Reading and preparing an input dataset ~~~~~~~~~~~~~~~~~
import numpy as np

input_field = np.load('field.npy')
R = 1.
thetas, phis = np.meshgrid(
    np.linspace(0, np.pi, input_field.shape[0]),
    np.linspace(-np.pi, np.pi, input_field.shape[1]),
    indexing='ij'
)
input_xs = R*np.sin(thetas)*np.cos(phis)
input_ys = R*np.sin(thetas)*np.sin(phis)
input_zs = R*np.cos(thetas)

# ~~~~~~~~~~ Using platerecipy ~~~~~~~~~~~

from platerecipy.model import PlateModel
from platerecipy.grid import SphericalGrid

input_xs    = input_xs    # to be specified ...
input_ys    = input_ys    # to be specified ...
input_zs    = input_zs    # to be specified ...
input_field = input_field # to be specified ...

# generating a consistent grid for interpolation
grid = SphericalGrid(input_xs, input_ys, input_zs)

# interpolating an input field
field = grid.interpolate_field(input_field, take_log=False)

# initializing a plate model
m = PlateModel(grid)

# stacking the interpolated field
m.stack_field(field, take_log=False)

# finding plates on the stacked field
m.find_plates(
    boundary_quantile      = 0.8,            # threshold for the boundaries
    separation_tolerance   = 2.5*np.pi/180., # 2 degrees for separation tolerance
                                             # patching 4-degree gaps
    RW_beta                = 50,             # RW beta (for feature sharpness)
```

```
    min_marker_size        = 0,              # to potentially filter out micro
    plates
    preserve_small_markers = True
)

# outputting as a ParaView readable .vtp file
from platerecipy import io
io.save_as_vtp(m, filename='basic.vtp')
```

5.2 examples/basic_custom_plot.py: A template from input to custom plots

```
"""
@file basic_custom_plots.py
@author Pejvak Javaheri; pejvak.javaheri@mail.utoronto.ca
@brief A template for basic usage of platerecipy in customized plots.
"""

# ~~~~~~~~~~~~~~~ Reading and preparing an input dataset ~~~~~~~~~~~~~~~~~
import numpy as np

input_field = np.load('field.npy')
R = 1.
thetas, phis = np.meshgrid(
    np.linspace(0, np.pi, input_field.shape[0]),
    np.linspace(-np.pi, np.pi, input_field.shape[1]),
    indexing='ij'
)
input_xs = R*np.sin(thetas)*np.cos(phis)
input_ys = R*np.sin(thetas)*np.sin(phis)
input_zs = R*np.cos(thetas)

# ~~~~~~~~~~ Using platerecipy ~~~~~~~~~~

from platerecipy.model import PlateModel
from platerecipy.grid import SphericalGrid

input_xs    = input_xs    # to be specified ...
input_ys    = input_ys    # to be specified ...
input_zs    = input_zs    # to be specified ...
input_field = input_field # to be specified ...

# generating a consistent grid for interpolation
grid = SphericalGrid(input_xs, input_ys, input_zs)

# interpolating an input field
field = grid.interpolate_field(input_field, take_log=False)

# initializing a plate model
m = PlateModel(grid)

# stacking the interpolated field
m.stack_field(field, take_log=False)

# finding plates on the stacked field
m.find_plates(
    boundary_quantile      = 0.8,            # threshold for the boundaries
    separation_tolerance   = 2.5*np.pi/180., # 2 degrees for separation tolerance
                                             # patching 4-degree gaps
    RW_beta                = 50,             # RW beta (for feature sharpness)
    min_marker_size        = 0,              # to potentially filter out micro
   plates
    preserve_small_markers = True
)

```

```python
# to create custom plots
import matplotlib.pyplot as plt

# a given plate
ID = 2

fig, ax = plt.subplots()
ax.set_title(
    f"Plate with plate ID = {ID}\n"
    + f'mean value across the plate = {np.mean(input_field[m.plate_IDs == ID]):7.4f
    }'
)
ax.imshow(m.plate_IDs == ID)
fig.savefig('basic_custom_plots.png', dpi=300)
```

6 Additional details

Function docstrings are to be considered the main source providing updated description. In order to access docstrings, one can easily invoke `help()` command or use `?` on `jupyter` notebook.

6.1 Overall structure

The source code is effectively divided to two portions:

- `src/platerecipy`: where the `Python` modules reside, and
- `src/clib`: where various backend utilities are implemented in `C` for better performance.

6.2 `segmentation` module

In `platerecipy`, Random Walker (RW) segmentation algorithm (Grady, 2006) is implemented primarily for the surface of a sphere (i.e., 3D spherical shell), though it is capable of performing segmentation on a flat 2D plane, as well. In short, linear system construction (i.e., vertex/edge associations, label ordering, etc.) is performed using the functions implemented in `C` (i.e., `segmentation.h`), while the numerical solution and the interface is done through the `Python` side (i.e., `segmentation.py`) using `NumPy` (Harris et al., 2020) and `SciPy` (Virtanen et al., 2020) routines.

The user interface is implemented by the function:

```
def random_walker(
    data                            : np.ndarray,
    labels                          : np.ndarray,
    beta                            = 100.,
    is_spherical                    = False,
    max_beta_reduction_attempts     = 3,
    beta_reduction_fraction         = 0.9,
    beta_reduction_residual_tol     = 1e-3
)
```

and returns the plate IDs and the ID probabilities as a tuple `(IDs, probs)`. Yet, function `random_walker` is effectively a wrapper that prepares the input to and output from functions `get_AB` and `solve_AX_B`. The `data` array is the input field/image and is assumed to be floating point. The `labels` must be of the same shape as `data` but an integer field with the unmarked regions labeled with zeros and each marked patch with a distinct but sequential positive integer label. Since overly large values for β destabilize the linear system, `random_walker` has a built-in mechanism that gradually reduces input `beta` by a factor of `beta_reduction_fraction`, `max_beta_reduction_attempts` many times until the L_2 norm of the residual of the solution to the numerical system is driven down below `beta_reduction_residual_tol`.

Function `get_AB` constructs the linear system and returns:

- `A`: the sparse L_{unmarked} matrix,
- `RHS`: the full right-hand-side matrix $-B^T \left[m^{(1)} m^{(2)} \cdots m^{(N)}\right]$, and
- `org2org`: the vector that maps the ordered node indices to the original indexing where `ord2org[i_ord] == i_org`.

whose values are subsequently passed onto `solve_AX_B` which accepts:

- `A`: the left-hand-side matrix,
- `B`: the right-hand-side matrix (essentially, `RHS` from `get_AB` and not to be confused with B in Grady, 2006) and must at least have two columns (otherwise its a trivial or an untenable solution),
- (optional) `solver`: by default set to LU decomposition (`'LU'`) but other options (e.g., `'direct', 'CG', 'FA'`) are available, and
- (optional) `safe_mode`: by default set to `True` which solves for all the probabilities (as opposed to solving for $N-1$ probabilities and assuming $p_N = 1 - \sum^{N-1} p_i$).

Finally, the solution to the linear system (which is ordered) is converted back to the original indexing using `ord2org`.

6.2.1 RW on a 2D plane

This mode is enabled by passing `is_spherical=False` and suggests that the input grid is a uniform rectilinear grid (effectively an image; see Figure 1).

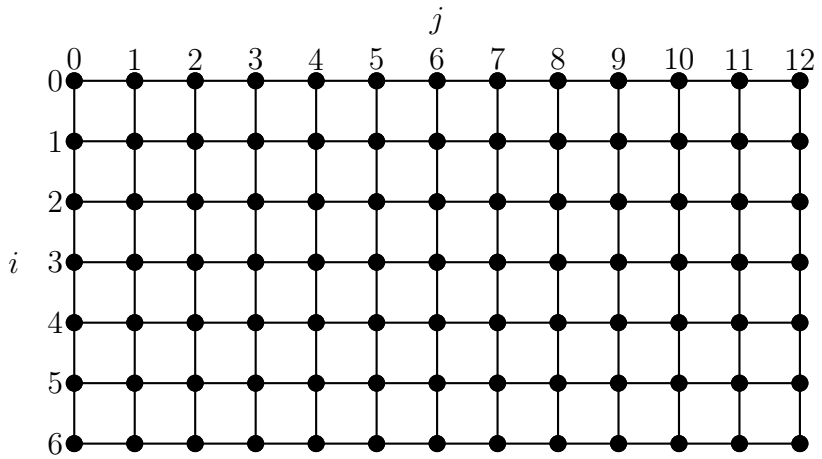

Figure 1: A sample 2D grid showing vertices and edges.

6.2.2 RW on a sphere

This mode is enabled by passing `is_spherical=True` and suggests that the input grid is a uniform spherical grid with wraparound boundary conditions. Since it is assumed that the `data` is an array containing the mercator projection of a sphere, to be consistent with the rest of the package, it is also assumed here that:

- `data[:, 0] == data[:, -1]` to satisfy wraparound boundary conditions (and similarly `labels` and the downstream output),
- `data[0, :]` are all the same and correspond to the north pole at $\theta = 0$, and

- `data[-1, :]` are all the same and correspond to the south pole at $\theta = \pi$.

Accordingly, at the entry point in `random_walker`, both `data` and `labels` are trimmed off of their redundant last columns, and all the helper `C` functions in `segmentation.h` that end with `_sph` assume the input arrays do not have the redundant column. This is critical as the arrays are passed by providing the pointer to the first index and the number of columns plays a fundamental role in the functions' ability to access, map, and order the indices. Furthermore, the last node on the first row and the first node on the last row of the trimmed array are considered as the north and south poles, respectively (see Figure 2).

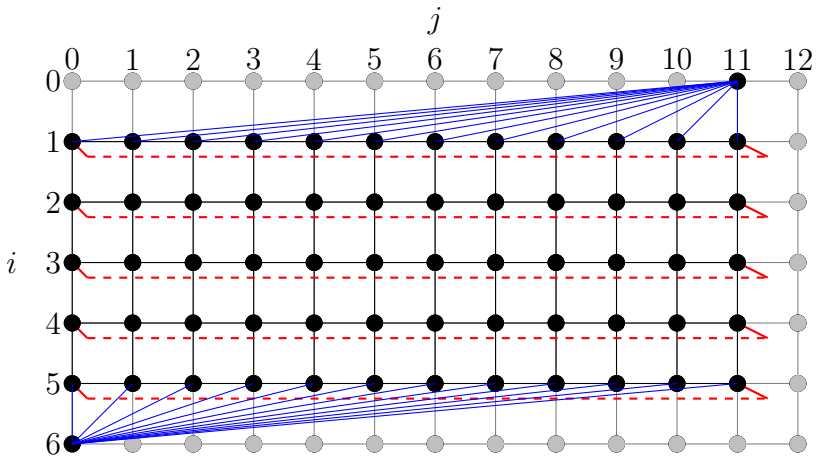

Figure 2: A sample spherical grid showing vertices and edges.

References

Grady, L. (2006). Random walks for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, *28*(11), 1768–1783.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

pandas development team, T. (2020). *Pandas-dev/pandas: Pandas* (Version latest). Zenodo. https://doi.org/10.5281/zenodo.3509134

Sullivan, C. B., & Kaszynski, A. (2019). PyVista: 3d plotting and mesh analysis through a streamlined interface for the visualization toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2